# Obelisk Proof Assistant — Reference Demonstration

Variant: Higher-Order Logic & Morse–Kelley Class Theory • Author: Georgy Dunaev • contact@obelisk.mozmail.com

## 1) Propositional logic

**1.01) Axm. A01 (Affirmation of Consequent):**

(α → (β → α))

Syntax side condition: α, β : Bool

**1.02) Axm. A02 (Distribution of Implication):**

((α → (β → γ)) → ((α → β) → (α → γ)))

Syntax side condition: α, β, γ : Bool

**1.03) Axm. A03 (Conjunction Elimination Left):**

((α ∧ β) → α)

Syntax side condition: α, β : Bool

**1.04) Axm. A04 (Conjunction Elimination Right):**

((α ∧ β) → β)

Syntax side condition: α, β : Bool

**1.05) Axm. A05 (Conjunction Introduction):**

(α → (β → (α ∧ β)))

Syntax side condition: α, β : Bool

**1.06) Axm. A06 (Disjunction Introduction Left):**

(α → (α ∨ β))

Syntax side condition: α, β : Bool

**1.07) Axm. A07 (Disjunction Introduction Right):**

(β → (α ∨ β))

Syntax side condition: α, β : Bool

**1.08) Axm. A08 (Disjunction Elimination / Cases):**

((α → γ) → ((β → γ) → ((α ∨ β) → γ)))

Syntax side condition: α, β, γ : Bool

**1.09) Axm. A09 (Negation Introduction / Contradiction):**

((¬α) → (α → ⊥))

Syntax side condition: α : Bool

**1.10) Axm. A10 (Reductio Ad Absurdum):**

((α → β) → ((α → ¬β) → ¬α))

Syntax side condition: α, β : Bool

**1.11) Axm. A11 (Ex Falso Quodlibet):**

(⊥ → α)

Syntax side condition: α : Bool

**1.12) Axm. A12 (Law of Excluded Middle):**

(α ∨ ¬α)

Syntax side condition: α : Bool

**1.13) Rule Modus Ponens (MP):**

α, (α → β) ⊢ β

Syntax side condition: α, β : Bool

## 2) Predicate logic

**2.01) Axm. Discourse domain is non-empty:**

∃(K : Class) ⊤

Syntax side condition: K : Class

**2.02) Rule Forall Instantiation (FI):**

((∀(x : Class) φ) → φ[t/x])

Syntax side condition: FFI(t, v_x, φ)

**2.03) Rule Exists Instantiation (EI):**

$(\varphi[t/x] \to \exists(x : \mathrm{Class})\ \varphi)$

Syntax side condition: $\mathrm{FFI}(t, v_x, \varphi)$

**2.04) Rule Forall Abstraction (FA / Bernays):**

$(\psi \to \varphi) \vdash (\psi \to \forall(x : \mathrm{Class})\ \varphi)$

Syntax side condition: $(\mathrm{NFV}(v_x, \psi) \land \mathrm{NFV}(v_x, \Gamma))$

**2.05) Rule Exists Abstraction (EA / Bernays):**

$(\varphi \to \psi) \vdash (\exists(x : \mathrm{Class})\ (\varphi \to \psi))$

Syntax side condition: $(\mathrm{NFV}(v_x, \psi) \land \mathrm{NFV}(v_x, \Gamma))$

**2.06) Rule Generalisation (Gen):**

$\varphi \vdash \forall(x : \mathrm{Class})\ \varphi$

Syntax side condition: $\mathrm{NFV}(v_x, \Gamma)$

## 3) Set/class-theoretic foundations of mathematics

**3.01) Axm. Class Comprehension Axiom:**

$\forall(X : \mathrm{Class})\ ((X \in \{\ x \mid \varphi(x)\ \}) \longleftrightarrow (\mathrm{isSet}(X) \land \varphi(X)))$

Syntax side condition: $\mathrm{NFV}(v_C, \varphi)$

**3.02) Axm. Extensionality Axiom:**

$\forall(A, B : \mathrm{Class})\ ((\forall(x : \mathrm{Class})\ ((x \in A) \longleftrightarrow (x \in B))) \to (A = B))$

Syntax side condition: $A, B, x : \mathrm{Class}$

**3.08) Axm. Heredity Axiom (Subclasses of Sets are Sets):**

$\forall(A, B : \mathrm{Class})\ ((\mathrm{isSet}(B) \land (A \subseteq B)) \to \mathrm{isSet}(A))$

Syntax side condition: $A, B : \mathrm{Class}$

**3.12) Axm. Regularity (Foundation) Axiom:**

$\forall(A : \mathrm{Class})\ ((A \neq \varnothing) \to \exists(x : \mathrm{Class})\ ((x \in A) \land ((x \cap A) = \varnothing)))$

Syntax side condition: $A, x : \mathrm{Class}$

**3.13) Axm. Pairing Axiom:**

$\forall(a, b : \mathrm{Class})\ ((\mathrm{isSet}(a) \land \mathrm{isSet}(b)) \to \mathrm{isSet}(\{a, b\}))$

Syntax side condition: $a, b : \mathrm{Class}$

**3.14) Axm. Union Axiom:**

$\forall(A : \mathrm{Class})\ (\mathrm{isSet}(A) \to \mathrm{isSet}(\bigcup A))$

Syntax side condition: $A : \mathrm{Class}$

**3.22) Def. IF-THEN-ELSE-FI (Class conditional with NFV side condition and context turnstile):**

$W, \Gamma \vdash (⟦\mathrm{IF}\ \psi\ \mathrm{THEN}\ A\ \mathrm{ELSE}\ B\ \mathrm{FI}⟧ = \{\ v \mid ((\psi \land (v \in A)) \lor (\neg\psi \land (v \in B)))\ \})$

Syntax side condition: $(\mathrm{NFV}(v_v, \psi) \land \mathrm{NFV}(v_v, \Gamma))$

**3.23) Def. simple_if (metalevel conditional over Type_Expression):**

$\forall(T_e : \mathrm{Type_Expression}),\ ◁(\mathrm{simple_if}(T_e) : (\mathrm{Bool} \to (⟦T_e⟧ \to (⟦T_e⟧ \to ⟦T_e⟧))))\ ▷$

$\forall(T_e : \mathrm{Type_Expression}),\ ◁(((⟦\mathrm{simple_if}(T_e, \mathrm{true}, t_1, t_2)⟧ = ⟦t_1⟧) \land (⟦\mathrm{simple_if}(T_e, \mathrm{false}, t_1, t_2)⟧ = ⟦t_2⟧))\ ▷$

Syntax side condition: $T_e : \mathrm{Type_Expression}, t_1 : ⟦T_e⟧, t_2 : ⟦T_e⟧$

**3.24) Def. dependent_if (dependent conditional over Bool → Type_Expression):**

$\forall(T_e : (\mathrm{Bool} \to \mathrm{Type_Expression})),\ ◁(\mathrm{dependent_if}(T_e) : \prod(b : \mathrm{Bool}).\ (⟦T_e(\mathrm{true})⟧ \to (⟦T_e(\mathrm{false})⟧ \to ⟦T_e(b)⟧)))\ ▷$

$◁(\mathrm{typeof}(⟦\mathrm{dependent_if}(T_e, b, t_1, t_2)⟧) := ⟦\mathrm{simple_if}(\mathrm{Type_Expression}, b, \mathrm{typeof}(t_1), \mathrm{typeof}(t_2))⟧)\ ▷$

$◁(((⟦\mathrm{dependent_if}(T_e, \mathrm{true}, t_1, t_2)⟧ = ⟦t_1⟧) \land (⟦\mathrm{dependent_if}(T_e, \mathrm{false}, t_1, t_2)⟧ = ⟦t_2⟧))\ ▷$

Syntax side condition: $T_e : (\mathrm{Bool} \to \mathrm{Type_Expression}), b : \mathrm{Bool}, t_1 : ⟦T_e(\mathrm{true})⟧, t_2 : ⟦T_e(\mathrm{false})⟧$

**3.32) Thm. Ordinal Induction (∈-Ind):**

$((\forall(\alpha : \mathrm{Class})\ ((\alpha \in \mathrm{On}) \to ((\forall(\beta : \mathrm{Class})\ ((\beta \in \alpha) \to \varphi(\beta))) \to \varphi(\alpha)))) \to \forall(\alpha : \mathrm{Class})\ ((\alpha \in \mathrm{On}) \to \varphi(\alpha)))$

Syntax side condition: $(\mathrm{Fresh}(v_\beta, v_\alpha) \land \mathrm{NFV}(v_\beta, \varphi(\alpha)))$

**3.33) Def. Power Class Definition (𝒫(a)):**

$\forall(a, x : \mathrm{Class})\ ((x \in 𝒫(a)) \longleftrightarrow (\mathrm{isSet}(x) \land (x \subseteq a)))$

Syntax side condition: $\mathrm{Fresh}(v_x, v_a)$

**3.34) Axm. Power Set Axiom (Power Class of a Set is a Set):**

$\forall(a : \mathrm{Class})\ (\mathrm{isSet}(a) \to \mathrm{isSet}(𝒫(a)))$

Syntax side condition: $a : \mathrm{Class}$

**3.37) Axm. Replacement / Limitation of Size Axiom:**
∀(F, A : Class) ((isClassFunction(F) ∧ isSet(A)) → isSet(F[A]))
Syntax side condition: F, A : Class

**3.40) Def. Dunaev Tagged Ordered Pair:**
∀(A, B : Class) ([A, B]_D = { z | ∃(a ∈ A) (z = {{{{a}}}, {∅}}) ∨ ∃(b ∈ B) (z = {{{{b}}}, {∅, {∅}}}) })
Syntax side condition: A, B : Class

**3.41) Thm. First Projection of Dunaev Tagged Ordered Pair:**
∀(A, B : Class) ({ a | ({{{{a}}}, {∅}} ∈ [A, B]_D) } = A)
Syntax side condition: A, B : Class

**3.42) Thm. Second Projection of Dunaev Tagged Ordered Pair:**
∀(A, B : Class) ({ b | ({{{{b}}}, {∅, {∅}}} ∈ [A, B]_D) } = B)
Syntax side condition: A, B : Class

**3.45) Axm. Infinity Axiom:**
∃(x : Class) (isSet(x) ∧ ((∅ ∈ x) ∧ ∀(y : Class) ((y ∈ x) → ((y ∪ {y}) ∈ x))))
Syntax side condition: x, y : Class

**3.46) Axm. Global Choice Axiom:**
∃(c : Class) (isClassFunction(c) ∧ ∀(x : Class) ((isSet(x) ∧ (x ≠ ∅)) → (c(x) ∈ x)))
Syntax side condition: c, x : Class

**3.47) Def. Universal Class (V):**
(V = { x | (x = x) })
Syntax side condition: V : Class

**3.53) Def. Transitive Class Predicate:**
(Tr(A) ⟷ ∀(x : Class) ((x ∈ A) → (x ⊆ A)))
Syntax side condition: A, x : Class

**3.66) Axm. Tarski's Axiom of Universes:**
∀(x : Class) (isSet(x) → ∃(u : Class) ((isUniverse(u) ∧ (x ∈ u)) ∧ isSet(u)))
Syntax side condition: x, u : Class

**3.67) Def. Semi-Singleton Class Predicate (semi_singleton):**
(semi_singleton(x) ⟷ ∀(u : Class) ∀(v : Class) (((u ∈ x) ∧ (v ∈ x)) → (u = v)))
Syntax side condition: x, u, v : Class

**3.68) Def. Value Extraction via Semi-Singleton Subclasses (sss / Value):**
(sss(x) = ⋃({ z | ((z ⊆ x) ∧ semi_singleton(z)) }))
Syntax side condition: x, z : Class

**3.69) Thm. Theorem of Evaluation of a Family ((Λx, t)(z) = t[z/x]):**
∀(z : Class) ((isSet(z) ∧ isSet(t[z/x])) → (((Λ(x : Class), t))(z) = t[z/x]))
Syntax side condition: (t : Class.Expression), (v_x : Variable), (z : Class)

**3.70) Def. Set-Theoretic Model of Dependent Sum (ΣB / Sigma Model):**
(ΣB = { p | ∃(x : Class) ∃(y : Class) (((x ∈ Dom(B)) ∧ (y ∈ B(x))) ∧ (p = [x, y]_D)) })
Syntax side condition: B, p, x, y : Class

**3.71) Def. Set-Theoretic Model of Dependent Product (ΠB / Pi Model):**
(ΠB = { f | ((isClassFunction(f) ∧ (Dom(f) = Dom(B))) ∧ ∀(x : Class) ((x ∈ Dom(B)) → (f(x) ∈ B(x)))) })
Syntax side condition: B, f, x : Class

## 4) Metalogical laws

**4.01) Axm. Instantiation Axiom:**
∀(T_e : Type_Expression), ◁ ∀(G : Ctx) ∀(φ : Fm) ∀(t : ⟦T_e⟧) ∀(v : Variable) ((W, G ⊢ (∀(v : ⟦T_e⟧) φ)) → (W, G ⊢ (let v = t in φ))) ▷
Syntax side condition: T_e : Type_Expression, t : ⟦T_e⟧, v : Variable

**4.02) Axm. Generalisation Axiom:**
∀(T_e : Type_Expression), ◁ ∀(Γ : Ctx) ∀(φ : Fm) ∀(v : Variable) (¬(Enc(v) ∈ Ctx.FV(Γ)) → ((W, Γ ⊢ φ) → (W, Γ ⊢ ∀(v : ⟦T_e⟧) φ))) ▷
Syntax side condition: NFV(v, Γ), T_e : Type_Expression

**4.03) Rule CoNecessitation:**
(F, [] ⊢ ‹›, Γ ⊢ φ›) ⊢ (F, Γ ⊢ φ)
Syntax side condition: F : Theory, Γ : Ctx, φ : Fm

**4.04) Def. Quantifier Unique (!):**
∀(φ : Fm) ((!(z : Class) φ) ⟷ ∀(x, y : Class) ((φ[x/z] ∧ φ[y/z]) → (x = y)))
Syntax side condition: z, x, y : Variable

**4.05) Def. Quantifier Exists and Unique (∃!):**
∀(φ : Fm) (T, [] ⊢ ((∃!(x : Class) φ) ⟷ ((∃(x : Class) φ) ∧ !(x : Class) φ)))
Syntax side condition: x : Variable, φ : Fm

**4.06) Axm. Class Comprehension Axiom (Metatheory):**
∀(φ : Fm) ∀(x : Name) ∀(k : Class.Expr) (FFI(k, x, φ) → (T, [] ⊢ ((k ∈ { x | φ }) ⟷ (isSet(k) ∧ φ[k/x]))))
Syntax side condition: FFI(k, x, φ)

**4.07) Axm. Soundness / Reflection Axiom:**
∀(φ : Fm) (⟦φ⟧ → (F, [] ⊢ φ))
Syntax side condition: φ : Fm, F : Theory

**4.08) Axm. Theory Inclusion (T ⊆ F):**
∀(Γ : Ctx) ∀(φ : Fm) ((T, Γ ⊢ φ) → (F, Γ ⊢ φ))
Syntax side condition: T, F : Theory, Γ : Ctx, φ : Fm

**4.09) Def. Syntactic Encoding Operator (Enc):**
∀(T : Type) ∀(s : T) (isSet(Enc(s)) ∧ (Enc(s) ∈ Emb(T)))
Syntax side condition: T : Type, s : T

**4.10) Def. Syntactic Decoding & Roundtrip Law (Dec):**
∀(T : Type) ∀(s : T) ((Dec(Enc(s), Emb(T)) : T) = (s : T))
Syntax side condition: T : Type, s : T

**4.11) Def. Dual Type Embedding Operator (Emb):**
∀(X : Class) ((Emb(Class)(X) ⟷ ⊤) ∧ (Emb(Set)(X) ⟷ isSet(X)))
Syntax side condition: X : Class

**4.12) Def. Proof-Object Class Family (ProofsOf):**
∀(s : Stmt) ∀(p : Prf) ((Enc(p) ∈ ProofsOf(Enc(s))) ⟷ CheckProof(p, s))
Syntax side condition: s : Stmt, p : Prf

**4.13) Def. Class of Provable Statements (Theorems):**
∀(s : Stmt) ((Enc(s) ∈ Theorems) ⟷ (∃(p : Class) (p ∈ ProofsOf(Enc(s)))))
Syntax side condition: s : Stmt, p : Class

## 5) Tactics and inference rules

**5.01) Thm. Elementary Theorems:**
((F, [(B ∈ x)] ⊢ ∃(x : Class) (B ∈ x)) ∧ (F, [(B ∈ x)] ⊢ isSet(B)))
Syntax side condition: B, x : Class

**5.02) Rule Definition to Introduction Rule:**
(F, [(B ∈ x)] ⊢ ∃(x : Class) (B ∈ x)) ⊢ (F, [(B ∈ x)] ⊢ isSet(B))
Syntax side condition: B, x : Class

**5.03) Rule Deduction Rule Application:**
(F, [] ⊢ ((B ∈ x) → isSet(B)))
Syntax side condition: B, x : Class

**5.04) Thm. Tactics Proof Example:**
F, [] ⊢ ((∃(y : Class) ∀(x : Class) (x ∈ y)) → ∀(x : Class) ∃(y : Class) (x ∈ y))
Syntax side condition: x, y : Class

## 6) Incompleteness-related statements

**6.01) Axm. Theories (Metalevel Stratification):**
((W ⊆ Q) ∧ ((Q ⊆ T) ∧ (T ⊆ F)))
Syntax side condition: W, Q, T, F : Theory

**6.02) Thm. Löb's Theorem:**

∀(φ : Fm) (Q, [] ⊢ ((T, [] ⊢ ((T, [] ⊢ φ) → φ)) → (T, [] ⊢ φ)))

Syntax side condition: φ : Fm

**6.03) Thm. Gödel's Second Incompleteness Theorem:**

Q, [] ⊢ (¬(T, [] ⊢ ⊥) → ¬(T, [] ⊢ ¬(T, [] ⊢ ⊥)))

Syntax side condition: T, Q : Theory

## Appendix: NBG Formulas-as-Types Standard Library

24 closed proof terms compiled by NBGSurfaceCompiler in Poly/ML. A formula is a type; a proof program is a term inhabiting it. The Dunaev pair injectivity theorem is the class-correct replacement for Kuratowski pairing on proper classes.

| Theorem | Formula (Type) | AST | Norm |
|---|---|---|---|
| empty_class_no_elements | (∀(z : Class) ((z ∈ { x \| ⊥ }) → ⊥)) | 7 | 7 |
| empty_class_is_subclass | (∀(X : Class) (∀(z : Class) ((z ∈ { x \| ⊥ }) → (z ∈ X)))) | 8 | 8 |
| universal_class_characterization | (∀(z : Class) (((z ∈ { x \| ⊤ }) → isSet(z)) ∧ (isSet(z) → (z ∈ { x \| ⊤ })))) | 15 | 15 |
| russell_class_is_not_a_set | (isSet({ x \| ((x ∈ x) → ⊥) }) → ⊥) | 19 | 32 |
| binary_inter_comm | (∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ { x \| ((x ∈ A) ∧ (x ∈ B)) }) → (z ∈ { x \| ((x ∈ B) ∧ (x ∈ A)) }))))) | 26 | 26 |
| binary_union_comm | (∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ { x \| ((x ∈ A) ∨ (x ∈ B)) }) → (z ∈ { x \| ((x ∈ B) ∨ (x ∈ A)) }))))) | 23 | 23 |
| class_diff_subclass_left | (∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ { x \| ((x ∈ A) ∧ ((x ∈ B) → ⊥)) }) → (z ∈ A))))) | 10 | 10 |
| class_eq_sym | (∀(X : Class) (∀(Y : Class) ((X = Y) → (Y = X)))) | 6 | 6 |
| class_eq_trans | (∀(X : Class) (∀(Y : Class) (∀(Z : Class) (((X = Y) ∧ (Y = Z)) → (X = Z))))) | 9 | 9 |
| quantifier_swap | ((∃(y : Set) (∀(x : Set) (x ∈ y))) → (∀(x : Set) (∃(y : Set) (x ∈ y)))) | 7 | 7 |
| dunaev_tagged_pair_projections | (∀(A : Class) (∀(B : Class) ((π₁([A, B]_D) = A) ∧ (π₂([A, B]_D) = B)))) | 3 | 3 |
| cantor_diagonal_theorem | (∀(R : Class) (∀(c : Set) (isSet(c) → ((((c ∈ { x \| ([x, x](_D ∈ R) → ⊥) }) → ([c, c]_D ∈ R)) ∧ ([c, c](_D ∈ R) → (c ∈ { x \| ([x, x](_D ∈ R) → ⊥) }))) → ⊥)))) | 28 | 50 |
| self_in_singleton | (∀(x : Class) (isSet(x) → (x ∈ { u \| (u = x) }))) | 8 | 8 |
| self_in_succ | (∀(x : Class) (isSet(x) → (x ∈ { u \| ((u ∈ x) ∨ (u = x)) }))) | 9 | 9 |
| class_sub_succ | (∀(x : Class) (∀(z : Class) ((z ∈ x) → (z ∈ { u \| ((u ∈ x) ∨ (u = x)) })))) | 12 | 12 |
| dunaev_pair_injective | (∀(A : Class) (∀(B : Class) (∀(C : Class) (∀(D : Class) (([A, B]_D = [C, D]_D) → ((A = C) ∧ (B = D))))))) | 26 | 26 |
| dunaev_cartesian_elim | (∀(A : Class) (∀(B : Class) (∀(p : Class) ((p ∈ { q \| (∃(u : Set) (∃(v : Set) (((u ∈ A) ∧ (v ∈ B)) ∧ (q = [u, v]_D)))) }) → (∃(u : Set) (∃(v : Set) (((u ∈ A) ∧ (v ∈ B)) ∧ (p = [u, v]_D)))))))) | 9 | 9 |
| inter_sub_left | (∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ { u \| ((u ∈ A) ∧ (u ∈ B)) }) → (z ∈ A))))) | 10 | 10 |
| inter_sub_right | (∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ { u \| ((u ∈ A) ∧ (u ∈ B)) }) → (z ∈ B))))) | 10 | 10 |
| union_sub_left | (∀(A : Class) (∀(B : Class) (∀(z : Class) ((z ∈ A) → (z ∈ { u \| ((u ∈ A) ∨ (u ∈ B)) })))) | 13 | 13 |
| semi_singleton_sub_elim | (∀(X : Class) (∀(z : Class) ((z ∈ { w \| ((w ∈ X) ∧ (∀(u : Class) (∀(v : Class) (((u ∈ X) ∧ (v ∈ X)) → (u = v))))) }) → (z ∈ X)))) | 9 | 9 |
| family_fiber_char | (∀(B : Class) (∀(x : Class) (∀(y : Class) ((y ∈ { w \| ([x, w]_D ∈ B) }) → ([x, y]_D ∈ B))))) | 8 | 8 |
| sigma_dependent_sum_intro | (∀(A : Class) (∀(B : Class) (∀(x : Class) (∀(y : Class) (((x ∈ A) ∧ ([x, y]_D ∈ B)) → ((x ∈ A) ∧ ([x, y]_D ∈ B))))))) | 8 | 8 |
| pi_dependent_product_elim | (∀(A : Class) (∀(B : Class) (∀(f : Class) (((∀(x : Class) ((x ∈ A) → ([x, f]_D ∈ B))) ∧ (∀(x : Class) (x ∈ A))) → (∀(x : Class) ([x, f]_D ∈ B)))))) | 10 | 10 |

Reproduce locally: python3 obelisk_cli.py compile examples/nbg_verified_programs.nbg